

ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Project Description

King Fahd University of Petroleum and Minerals (KFUPM)

Course: ICS-344

Term: 252

Title: Information Security

Date Assigned: 14 March 2026

Instructor:

Due date: 19 April 2026

Introduction

This course project is designed to give ICS-344 students a practical experience in discovering, exploiting, documenting, and fixing security weaknesses in a realistic cloud application by using DVSA, the OWASP Damn Vulnerable *Serverless* Application, in a controlled learning environment. By working with an AWS-based deployment that includes an S3-hosted frontend, API Gateway, Lambda functions, and backend services such as DynamoDB and related cloud resources, you will connect core serverless computing concepts with real security risks, including broken authentication, access control flaws, insecure cloud configuration, over-privileged IAM roles, input-handling issues, and logic errors. The overall purpose is not only to reproduce the official security lessons, but also to develop the ability to analyze attack surface, understand cloud-specific impact and blast radius, and apply secure design principles such as validation, least privilege, and defense in depth when remediating vulnerabilities in modern AWS serverless systems.

Objectives

The primary objective of this course project is to provide hands-on practice in identifying, exploiting, documenting, and remediating common serverless security vulnerabilities in DVSA. You are encouraged to explore the application systematically and discover both the official vulnerabilities covered in the lessons and any additional undocumented issues you may find. As part of the core task, the DVSA environment must be safely deployed in a non-production AWS account, and the ten official vulnerabilities must be demonstrated, supported with evidence, fixed appropriately, and verified after remediation. For each vulnerability, the report must begin by explaining the **intended logic of the relevant operation or workflow**. For instance, if the case involves an order request, the report should describe how the request is expected to flow through the system, which functions or cloud services participate in processing it, what validation or authorization checks should occur, and what the correct outcome should be. The report must then identify the **security weakness**, describe the **exploit path**, and analyze the resulting **impact on the application**. Where appropriate, students should support this explanation with clear visual representations, such as **sequence diagrams**, **data-flow diagrams**, **workflow diagrams**, or similar architectural figures. You must then submit a structured report that clearly presents the reproduction steps, proof of exploit, exact code or configuration changes, and post-fix verification. Additional valid findings may be included for bonus credit.

High-Level Overview of Serverless Architecture

In a traditional architecture, an application runs on servers that are always on, even when no one is using the application. This means the system owner is responsible for setting up those servers, updating them, patching them, scaling them, and keeping them available. In a **serverless** architecture, developers do not manage the servers directly. Instead, the cloud provider automatically provides the computing resources only when they

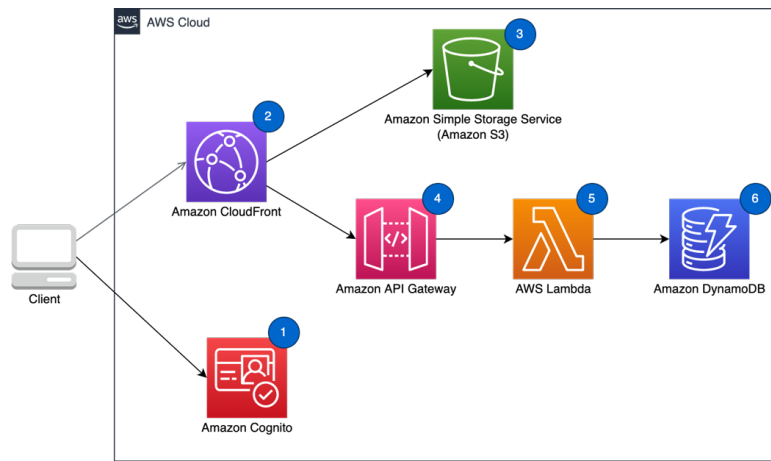


Figure 1: Serverless web application architecture

are needed. Developers mainly focus on writing the application code and defining how different cloud services should work together.

A simple way to understand serverless is to think of it as an **event-driven model**. An event happens, and the cloud reacts automatically. For example, a user may open a website, log in, submit an order, upload a file, or request some data. That action triggers a cloud service, which then calls the necessary backend logic. The backend processes the request, stores or retrieves data, and returns a response. The servers still exist in the background, but AWS manages them automatically, so the developer does not have to run or maintain them manually.

For this project, you will work with the Damn Vulnerable Serverless Application (DVSA), which runs entirely on AWS. A beginner-friendly way to think about DVSA is as follows: the user opens the web interface, sends a request, AWS forwards that request to a backend function, the function processes the request, interacts with storage or messaging services if needed, and then sends the result back to the user. In DVSA, the frontend is a static website hosted on Amazon S3, while the backend uses Amazon API Gateway and AWS Lambda. Data and application state are stored using DynamoDB tables, S3 buckets, and SQS queues. Understanding this flow is important because many serverless vulnerabilities appear in the way these services interact, not only in the code itself.

The provided figure in 1 shows a common AWS serverless web application design. In this architecture, **Amazon Cognito (1)** manages user authentication and identity. This means it helps users sign in and allows the application to verify who they are. Static web content such as HTML, CSS, JavaScript, and images may be delivered through **Amazon CloudFront (2)** from an **Amazon S3 (3)** bucket. When the user performs an action that needs backend processing, the request is sent to **Amazon API Gateway (4)**, which acts as the public entry point for the application's APIs. API Gateway then invokes an **AWS Lambda (5)** function, which contains the actual application logic. The Lambda function may validate input, check permissions, process the request, and read from or write to **Amazon DynamoDB (6)** for persistent storage. This combination of services is a common pattern in AWS serverless applications.

As a simple example, consider an online order request. First, the user opens the website hosted from S3 and CloudFront. If authentication is required, the user signs in through Cognito. Next, the user submits an order through the web interface. That request is sent to API Gateway, which forwards it to a Lambda function. The Lambda function checks whether the request is valid, verifies that the user is allowed to perform the action, processes the order, and stores the order details in DynamoDB. If needed, it may also place a message in SQS for later background processing. Finally, the result is returned to the frontend and shown to the user. This step-by-step logic is important because security problems can occur at each stage, such as weak authentication, missing authorization checks, unsafe input handling, or overly broad permissions between services.

0.1 The Serverless Pipeline and Request Lifecycle

To understand vulnerabilities in DVSA, such as code injection, broken authentication, access control flaws, or unsafe backend data handling, you must first understand how data moves through the application. When a user interacts with the DVSA frontend, an HTTP request is sent from the browser. This request reaches Amazon API Gateway, which acts as the public entry point to the backend. API Gateway then routes the

request to a specific AWS Lambda function. The Lambda function executes the required business logic, such as validating a JWT, processing an order, or calculating a cart total, and may interact with backend services such as Amazon DynamoDB, Amazon S3, Amazon SES, or Amazon SQS. Finally, the Lambda function returns a response through API Gateway back to the user's browser. Understanding this request lifecycle is essential for identifying where security weaknesses may appear, such as in input handling, authentication, authorization, service permissions, or backend data access.

Note: Although normal users typically generate these requests through the browser frontend, security testing may also be performed by sending HTTP requests directly to the same API Gateway endpoint using tools such as curl or Postman. In both cases, the serverless pipeline remains the same: the request still enters through API Gateway, is routed to a Lambda function, and is then processed by the backend services. Direct API testing is simply a more controlled way to study how the backend handles malicious, malformed, or unexpected input.

0.2 Amazon API Gateway (The Entry Point)

Amazon API Gateway is a fully managed AWS service for creating, publishing, monitoring, and securing APIs. In DVSA, API Gateway acts as the main entry point for backend functionality. It defines the public routes and HTTP methods that the application exposes, such as endpoints for orders, carts, or account-related actions. API Gateway receives client requests and forwards them to the appropriate Lambda functions. Because it is the first backend component to handle incoming requests, it is an important part of the attack surface. Security issues at this layer may include weak route protection, improper request validation, or unsafe handling of user-controlled input before the request reaches the application logic.

0.3 AWS Lambda (Compute Layer)

AWS Lambda is an event-driven serverless compute service that runs code in response to events and automatically manages the underlying infrastructure. In DVSA, Lambda functions implement the backend business logic of the application. For example, a Lambda function may validate a request, check whether a user is authorized, calculate prices, send notifications, or store data. Lambda functions execute in short-lived managed environments and are given temporary AWS permissions through their execution roles. Because Lambda contains the application's core logic, vulnerabilities at this layer may include insecure input handling, missing authorization checks, logic flaws, or misuse of cloud permissions.

0.4 Amazon DynamoDB (Data Persistence)

Amazon DynamoDB is a fully managed NoSQL database service commonly used in serverless applications to store persistent application data. In DVSA, DynamoDB may be used to store information such as user records, orders, carts, or other backend state. Since many serverless applications rely heavily on managed data stores, it is important to understand how data is written, retrieved, and trusted across different parts of the application. Even though DynamoDB is not a relational database and is not subject to traditional SQL injection in the usual sense, insecure handling of stored data can still lead to serious security problems, especially if data entered at one stage is later reused unsafely by another component or workflow.

0.5 Identity and Access Management (IAM) and Execution Roles

AWS Identity and Access Management (IAM) controls access to AWS services and resources. In serverless applications, each Lambda function runs with an execution role that defines what AWS actions the function is allowed to perform. For example, a function may be allowed to read from an S3 bucket, write to a DynamoDB table, or send a message to an SQS queue. A common serverless security problem is the over-privileged function, where the execution role grants more permissions than the function actually needs. If such a function is compromised or abused, the attacker may inherit those excessive permissions and use them to expand the impact of the attack. For this reason, serverless functions should follow the Principle of Least Privilege.

0.6 Authentication: Amazon Cognito and JWTs

Modern serverless applications often use identity providers such as Amazon Cognito for authentication and user management. When a user logs in, Cognito can issue a JSON Web Token (JWT), which contains information about the authenticated user. A JWT typically includes a header, a payload with claims such as

the user identity, and a cryptographic signature. Before trusting the token, the application must verify that the signature is valid and that the token was issued by a trusted source. If the application fails to verify the token correctly, an attacker may be able to modify claims and impersonate another user. This kind of weakness can lead to broken authentication and unauthorized access.

0.7 Observability: CloudWatch and CloudTrail

In serverless applications, monitoring and logging are essential because backend processing is distributed across multiple managed services. Amazon CloudWatch collects operational data such as logs and metrics. In DVSA, Lambda function outputs such as `console.log` or `print()` are typically stored in CloudWatch Logs, making it a key place to inspect request handling, debug failures, and verify whether a vulnerability has been reproduced or fixed. AWS CloudTrail records API activity across the AWS account, including which actions were performed and which IAM identities performed them. This is useful for understanding the real behavior of the application, investigating permission usage, and helping determine whether an execution role should be narrowed during remediation.

0.8 Observability: CloudWatch and CloudTrail

To effectively exploit and debug serverless applications, you must rely on AWS logging services since API Gateway often hides backend crashes behind generic 500 errors. Amazon CloudWatch automatically collects monitoring and operational data. Lambda logs all requests and stores `console.log` or `print()` outputs in CloudWatch Logs, making it the primary place to verify if your code injection or data exfiltration succeeded. AWS CloudTrail, conversely, logs all API activity across your AWS account. It records exactly what actions an IAM role actually performed, which is essential for utilizing the IAM Policy Simulator and determining the minimum required permissions to fix an over-privileged role.

About DVSA

The OWASP DVSA (Damn Vulnerable Serverless Application) is an official, open-source project hosted by the OWASP Foundation. It is a deliberately vulnerable application aiming to be an aid for security professionals to test their skills and tools in a legal environment, help developers better understand the processes of securing serverless applications and to aid both students & teachers to learn about serverless application security in a controlled classroom environment. The aim of DVSA is to practice some of the most common serverless vulnerabilities, with a simple straightforward interface. Please note, there are both documented and undocumented vulnerabilities with this software. This is intentional. You are encouraged to try and discover as many issues as possible. Do not install DVSA on a production account. No one will take responsibility for the way in which anyone uses this application (DVSA). The purpose of the application is demonstration only and it should not be used maliciously or in any production accounts.

Copyright and Usage Notice

DVSA is intentionally vulnerable. It must be deployed **only** in a **non-production AWS account**. It is for legal classroom demonstration only and must never be used maliciously. This project along with its materials and techniques, is the intellectual property of KFUPM/Course Instructors- semester 252 and is intended solely for educational purposes within the ICS-344: Information Security Course. The content aims to foster hands-on learning in offensive security and defense strategies. Any unauthorized use, distribution, reproduction, or adaptation of this project, including the methods and tools described, outside the context of this course, especially for malicious or non-educational purposes is strictly prohibited. Written permission from the instructor or institution is required for any use beyond this ICS-344.

ICS-344 Course Project: DVSA Vulnerability Discovery and Remediation

Project Task

In this project, you will deploy the OWASP DVSA (Damn Vulnerable Serverless Application) in a **non-production AWS account**, study the **10 official vulnerability lessons**, and then systematically identify, exploit, document, fix, and verify vulnerabilities in the application. For each official vulnerability, you are required to provide clear reproduction steps, proof of the issue, the exact code or configuration changes used to remediate it, and evidence showing that the fix works without breaking normal functionality. *For each case, you must first explain the intended logic of the relevant operation or workflow, including the expected request flow, the AWS services involved, and the checks that should normally occur. You should then explain the exploit path, the resulting security impact and blast radius, and how the fix applies secure design principles such as input validation, least privilege, and defense in depth. Where helpful, you may use sequence diagrams, data-flow diagrams, workflow diagrams, or similar figures to make the analysis clearer.* In addition to the official lessons, you are encouraged to discover and document **additional undocumented vulnerabilities**, since bonus findings can improve your grade. DVSA is intentionally vulnerable and is provided strictly for legal, educational, and demonstration purposes only; it must not be used maliciously or deployed in any production environment.

Scope of the Assignment

Please note that these lessons cover only a small portion of the vulnerabilities that exist in DVSA. Some of these are already demonstrated in the project document. There are dozens more vulnerabilities and scenarios that are not covered in the lessons, and you are encouraged to find and document more vulnerabilities that you discover.

The more vulnerabilities you find and fix, the more your chances increase to earn a better grade.

Important Update Note

Some exploits may not behave exactly as described due to AWS Lambda runtime / SDK updates over time. It is still worth trying each lesson.

If you discover that a lesson's exploit is no longer a vulnerability because of an update or another reason, you must still:

- include a screenshot showing the observed behavior, and
- explain in detail, step by step, how you tested it and how you concluded that it is no longer vulnerable.

The 10 Official Vulnerability Lessons (with link)

- | | |
|--|--|
| ▶ LESSON #1: Event Injection | ▶ LESSON #2: Broken Authentication |
| ▶ LESSON #3: Sensitive Data Exposure | ▶ LESSON #4: Insecure Cloud Configurations |
| ▶ LESSON #5: Broken Access Control | ▶ LESSON #6: Denial of Service (DoS) |
| ▶ LESSON #7: Over-Privileged Functions | ▶ LESSON #8: Logic Vulnerabilities |
| ▶ LESSON #9: Vulnerable Dependencies | ▶ LESSON #10: Unhandled Exceptions |

1 Brief Overview of the 10 Official Vulnerability Lessons

Lesson #1: Event Injection

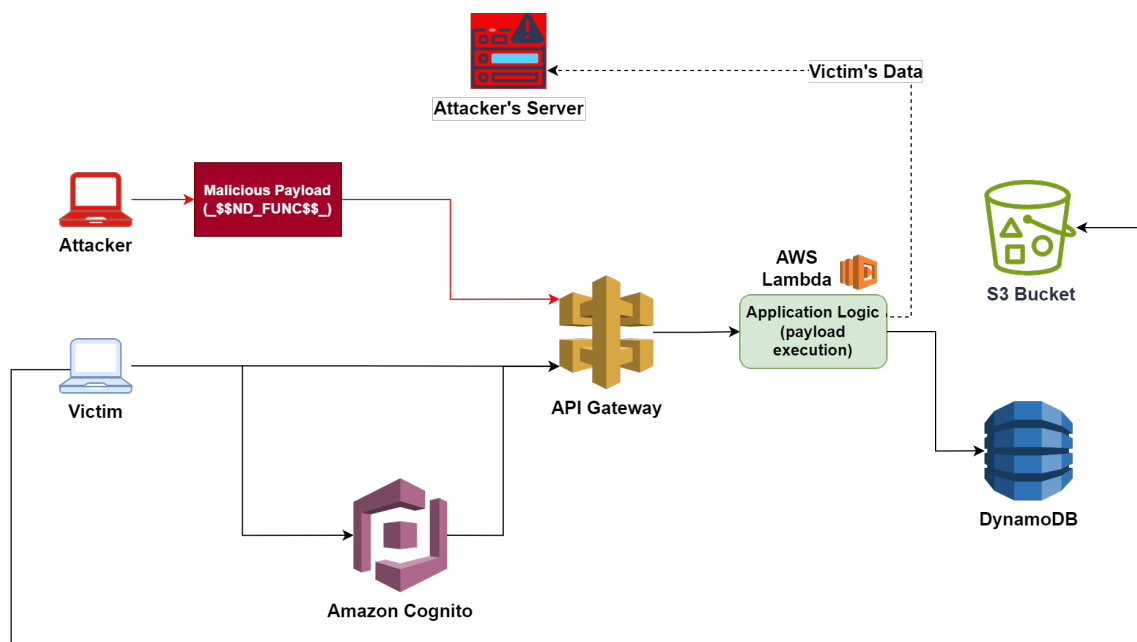


Figure 2: Lesson #1: Event Injection

Figure 2 illustrates the event injection lesson, where attacker-controlled input is sent through the API path and is processed unsafely by the backend Lambda function. You should demonstrate that request data is treated as executable content rather than plain data, identify the insecure deserialization or dynamic execution point in the API-handling function, and show its security impact, such as invoking unintended functionality or exposing other users' data. The vulnerability exists in the request-processing logic, and the appropriate fix is to remove unsafe deserialization or code evaluation, validate request fields strictly with an allowlist, and ensure the API entry function cannot execute arbitrary user-controlled content.

Lesson #2: Broken Authentication

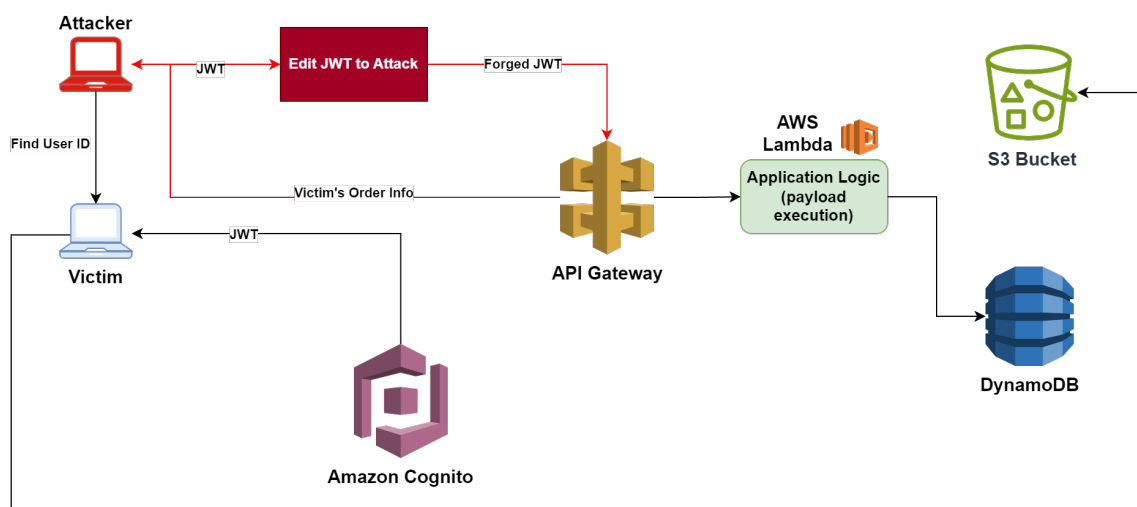


Figure 3: Lesson #2: Broken Authentication

Figure 3 shows the main broken authentication scenario, where a JSON Web Token (JWT) can be modified and reused to access another user's order information. In this lesson, you should verify how the application trusts token contents without enforcing proper integrity checks, and also examine whether sensitive billing functionality is exposed without adequate authentication. The weakness lies in the authentication and authorization checks performed at the API layer, and the fix is to verify JWT signatures and claims correctly on the server side, reject forged or altered tokens, and require valid authentication on every protected endpoint.

Lesson #3: Sensitive Information Disclosure

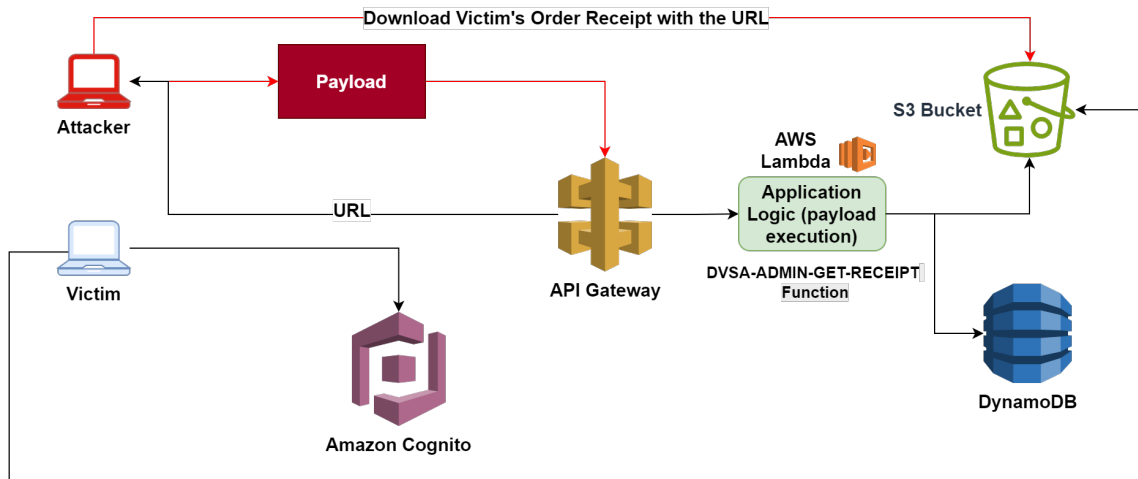


Figure 4: Lesson #3: Sensitive Information Disclosure

Figure 4 represents the sensitive information disclosure lesson, where privileged backend functionality can be abused to obtain access to receipt files stored in Amazon S3. You should show that receipt-related information or download links can be generated outside the intended authorization boundary and then used to access data that should remain restricted. The vulnerability is rooted in weak access control around administrative receipt operations and file exposure, and the likely fix is to enforce strict authorization on receipt-generation functions, limit who can obtain signed URLs, and reduce unnecessary exposure of S3-hosted sensitive files.

Lesson #4: Insecure Cloud Configuration

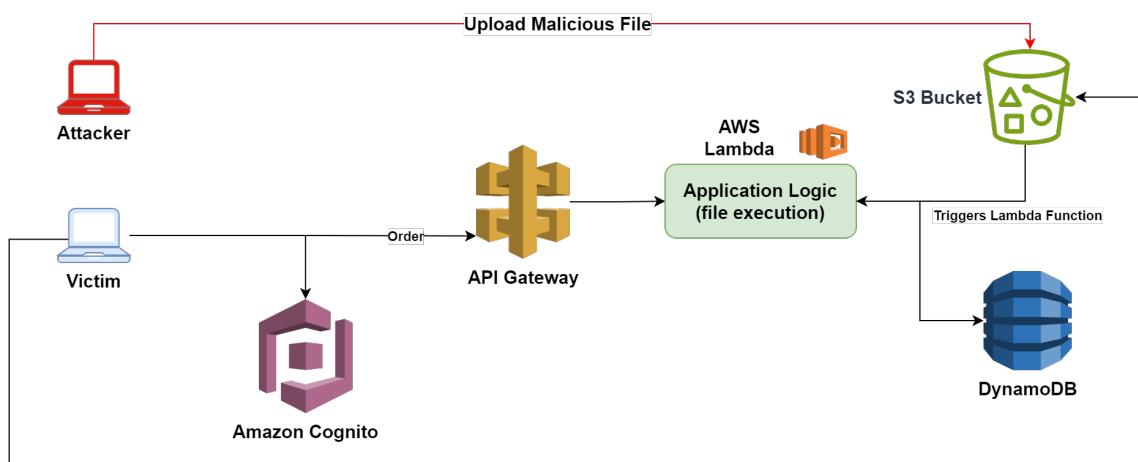


Figure 5: Lesson #4: Insecure Cloud Configuration

Figure 5 illustrates an insecure cloud configuration in which a malicious file can be uploaded directly into the S3 bucket and later processed by a Lambda function. You should demonstrate that the bucket permissions are too permissive and that downstream processing of attacker-controlled objects or filenames creates a path

to unintended code or command execution. The weakness is therefore both a storage misconfiguration and an unsafe backend processing flow, and the fix is to harden the S3 bucket policy, remove unintended public or cross-account write access, validate uploaded object names and types, and avoid executing commands that incorporate user-controlled file data.

Lesson #5: Broken Access Control

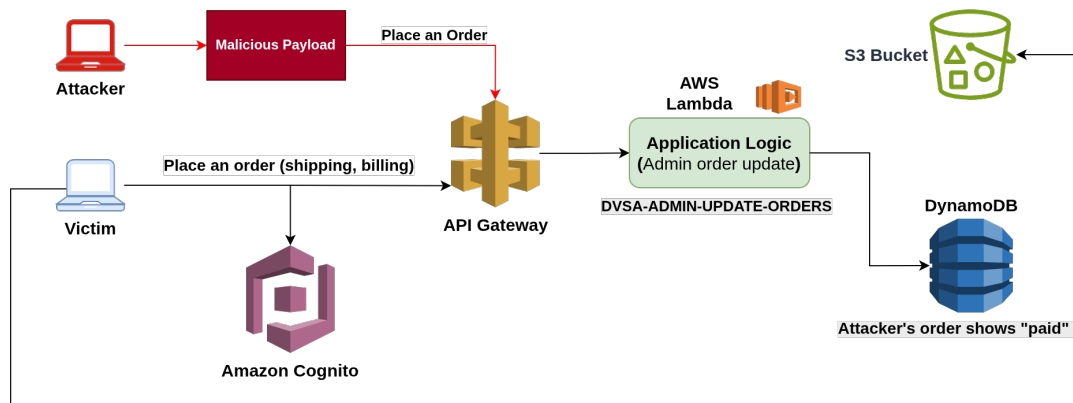


Figure 6: Lesson #5: Broken Access Control

Figure 6 shows the broken access control lesson, where an attacker abuses the vulnerable API path to invoke the administrative function DVSA-ADMIN-UPDATE-ORDERS and update an order outside the intended payment workflow. You should demonstrate that a privileged backend operation can be reached from an unprivileged context, allowing the order status to be changed so that it appears paid without completing the normal billing process. The vulnerability lies in missing authorization checks around sensitive administrative functionality, and the fix is to enforce strict role-based access control, validate order state transitions on the server side, and ensure that public-facing functions cannot invoke privileged internal actions without explicit authorization.

Lesson #6: Denial of Service (DoS)

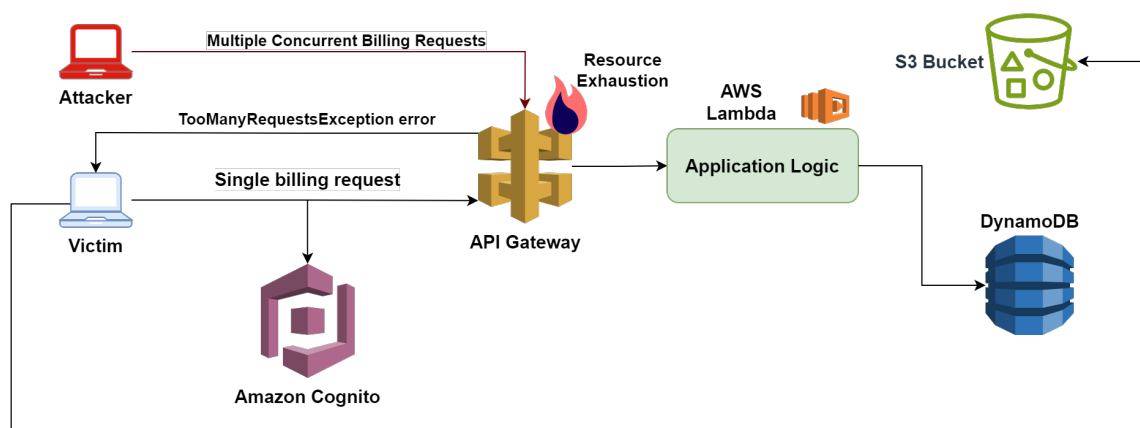


Figure 7: Lesson #6: Denial of Service (DoS)

Figure 7 illustrates how repeated concurrent billing requests can consume the limited processing capacity of the billing path and prevent legitimate users from completing payment. In this lesson, you should demonstrate that the billing service can be saturated by parallel requests, causing throttling or unavailability for normal checkout activity. The vulnerability is located in the resource protection and request-handling design rather than in data confidentiality, and the likely fix is to apply rate limiting, add abuse protection at the API layer, isolate critical concurrency, and introduce fair request controls such as queues or per-user throttling.

Lesson #7: Over-Privileged Function

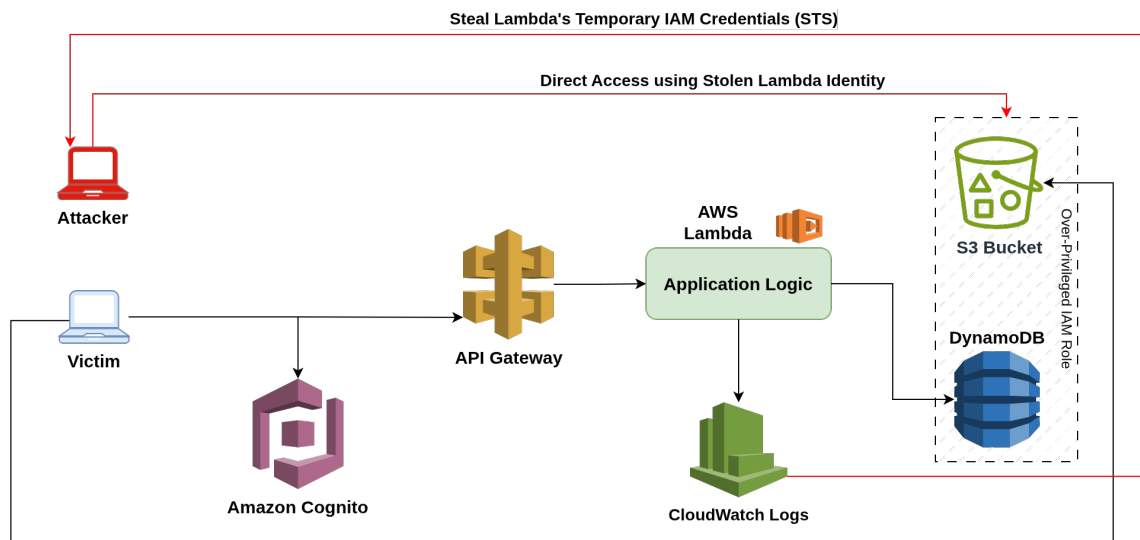


Figure 8: Lesson #7: Over-Privileged Function

Figure 8 illustrates the over-privileged function lesson, where an attacker first obtains the Lambda function's temporary STS credentials and then uses them to access AWS resources beyond the function's legitimate purpose. You should show that compromise of the function effectively exposes the permissions of its execution role, enabling unauthorized access to services such as Amazon S3 and Amazon DynamoDB. The vulnerability is therefore not only the initial code execution path, but also the excessive IAM permissions attached to the function role. The appropriate fix is to apply the principle of least privilege by restricting the role to only the exact S3 objects, DynamoDB tables, and other AWS actions required for the function's intended task.

Lesson #8: Logic Vulnerabilities

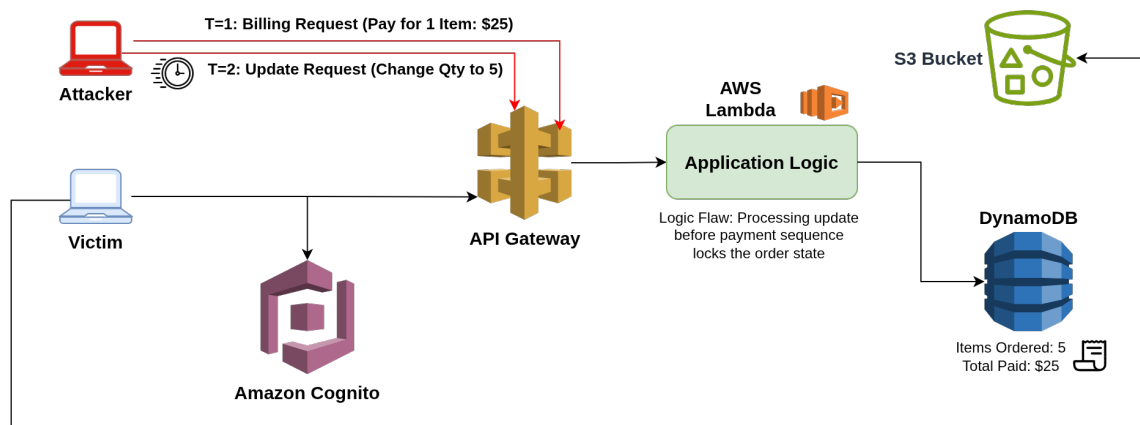


Figure 9: Lesson #8: Logic Vulnerabilities

Figure 9 illustrates the logic vulnerability lesson as a race condition in the order-processing workflow. You should demonstrate that if a billing request is submitted and an update request is sent immediately afterward, the application may process the requests out of sequence, allowing the final order state to differ from what was actually paid for. The weakness is not a simple input-validation flaw, but a business logic error in how order state changes are coordinated across requests. The likely fix is to enforce strict server-side workflow sequencing, prevent order modification once billing begins, and use locking or conditional updates so that billing and order-update operations cannot be abused through timing.

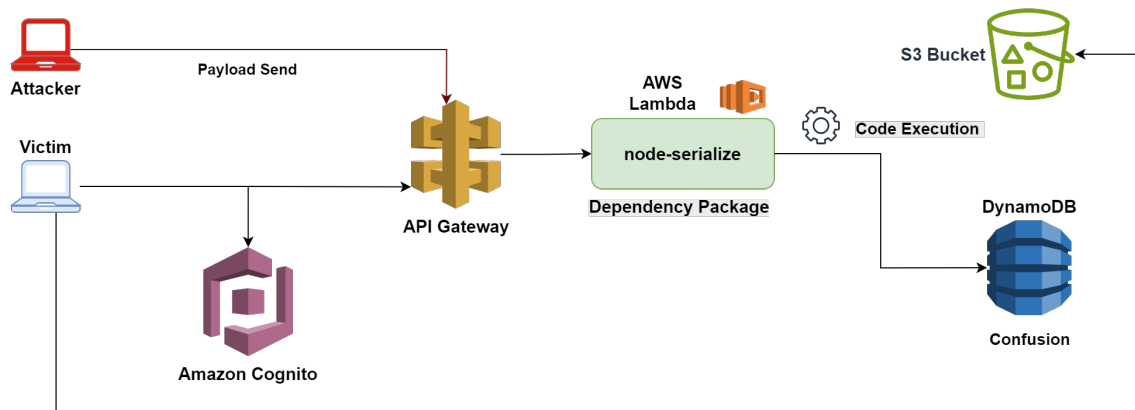


Figure 10: Lesson #9: Vulnerable Dependencies

Lesson #9: Vulnerable Dependencies

Figure 10 highlights that insecure third-party packages inside the Lambda deployment can become the real entry point for compromise. You should identify the risky dependencies used by the API-handling backend, relate them to the observed attack surface, and explain how a vulnerable package such as `node-serialize` can enable dangerous behavior such as code execution. The weakness is therefore in dependency selection and maintenance, and the fix is to remove or replace unsafe libraries, upgrade to supported alternatives, pin reviewed versions, and continuously scan dependencies during development and deployment.

Lesson #10: Unhandled Exceptions

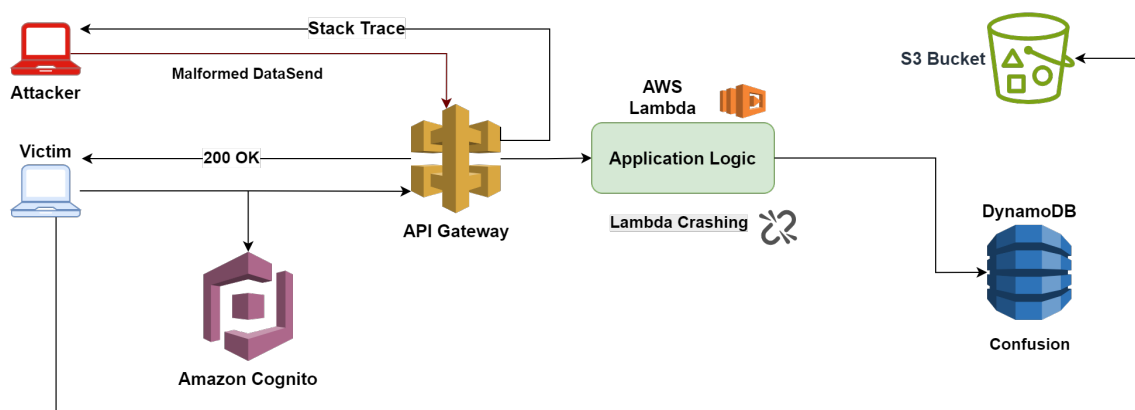


Figure 11: Lesson #10: Unhandled Exceptions

Figure 11 shows how malformed or incomplete requests can cause backend errors that are returned to the client with excessive detail. You should demonstrate that the application leaks stack traces, internal file paths, code fragments, or backend validation details when exceptions are not handled safely. The vulnerability lies in poor error handling across the Lambda and API integration path, and the fix is to catch exceptions centrally, validate input before deeper processing, return only generic client-safe error messages, and keep detailed diagnostics only in internal logging systems such as CloudWatch.

Helper Files are Available

In the project files, you will find a **helper Guide** that demonstrates a few of the lessons in details. There is also recorded videos for one lesson and its remediation. Besides, another video shows how to setup the AWS console and how to deploy DVSA.

Submission Guideline

Use the following format to report *each* vulnerability you demonstrate, analyze, and fix. Follow the structure below so that your work is clear, reproducible, and consistent with the style used in the demonstration lessons.

Template for Report

For every vulnerability/lesson, use the structure below.

0) Cover Page

- **Course / Section:** _____
- **Student Name(s) + ID(s):** _____
- **Your DVSA Website URL:** _____
- **AWS Region:** _____
- **Date:** _____
- **Vulnerability Title(s):** _____

For Each Vulnerability (repeat the sections below)

Repeat the **10 parts** below for every vulnerability (lesson). You may present the exploit demonstration and the fix demonstration either in one continuous subsection or in two separate subsections, provided that the required headings remain clear. If a single exploit naturally demonstrates two closely related lessons, you may combine them into one subsection, but you must explicitly identify both lessons and explain both. For guidance, please refer to the helper guide, which includes a sample showing how these 10 parts can be written for Lesson #2: Broken Authentication.

Part 1) Goal and Vulnerability Summary

Briefly introduce the lesson in one short paragraph. Your summary should state:

- the vulnerability name or type,
- the main affected component (for example API Gateway, Lambda, IAM role, S3, DynamoDB, or JWT handling),
- the security impact,
- and the main weakness at a high level.

This section should help the reader understand what the lesson is about before the detailed steps begin.

Part 2) Why This Works / Root Cause

Explain briefly why the vulnerability is possible. Focus on the security mistake that makes the attack succeed, such as trusting unverified JWT claims, unsafe deserialization, missing authorization checks, weak workflow logic, or over-privileged IAM permissions. Keep this section short and conceptual.

Part 3) Environment and Setup

State only the setup details that are needed to understand or reproduce the lesson. Include relevant items such as:

- the DVSA URL or API endpoint used,
- the AWS service, Lambda function, IAM role, or CloudWatch log group involved,
- the account or user context used in the test, if relevant,
- and any tools used, such as browser DevTools, `curl`, `jq`, `python3`, AWS Console, or AWS CLI.

Only include setup details that are actually used in your demonstration.

Part 4) Reproduction Steps

Write clear numbered steps that another student could follow. Use the same style as the demonstrations:

- describe the exact action taken,
- include the relevant command if you used `curl`, `aws`, or a helper script,
- and keep the order of steps logical and easy to follow.

If you include tokens, secrets, or identifiers, redact sensitive values where appropriate.

Part 5) Evidence and Proof

Show what proves that the vulnerability is real. This may include:

- a screenshot of the browser request or response,
- a screenshot of CloudWatch logs,
- a screenshot of IAM Policy Simulator or AWS Console output,
- terminal output from `curl`, `jq`, or AWS CLI,
- or a short explanation of what result confirms the vulnerability.

You do not need to force every lesson into the same type of evidence; use the kind of proof that best matches the lesson.

Part 6) Fix Strategy / Probable Mitigation

Briefly explain how the vulnerability should be fixed. State:

- where the fix belongs (for example a Lambda function, IAM policy, validation logic, or workflow check),
- what type of security improvement is needed,
- and why that mitigation addresses the root cause.

This section is about the intended security improvement, even if the full fix is shown later.

Part 7) Code / Config Changes

Show the actual fix you applied. Include what is relevant for your case, such as:

- a code snippet,
- a before/after comparison,
- a short diff,
- or a screenshot of the modified code, policy, or configuration.

Also identify:

- the file path, function name, or AWS resource changed,
- and the main logic or permission that was added, removed, or restricted.

Part 8) Verification After Fix

Repeat the relevant test and show that the vulnerable behavior no longer succeeds. Include:

- the post-fix result,
- one short statement explaining what changed,
- and, where appropriate, a quick confirmation that the intended legitimate behavior still works.

For example, a forged token should now fail, a dangerous request should no longer execute, or an overly broad permission should no longer be allowed.

Part 9) Structured Operation and Security Analysis

For each **vulnerability**, you must complete an additional structured analysis. The purpose of this analysis is to make your report more rigorous by clearly distinguishing between:

1. how the application is **supposed** to behave,
2. how it **actually** behaved during the exploit,
3. why that behavior represents a **security deviation**, and

4. how the **fix restores the correct behavior**.

For each selected vulnerability, you must submit the following **four things**:

1) Intended Logic and Security Rule(s). Explain the intended logic of the relevant operation or workflow under normal conditions. Identify the main actor, the API or entry point, the AWS services involved, the expected validation or authorization checks, and the correct output or outcome. Then state one or more short **security rules** that describe what the system must or must not do.

2) Evidence Sources and Behavior Trace. Identify the main sources you used to understand the intended logic and the actual behavior. These may include browser workflow, API requests and responses, decoded JWT contents, CloudWatch logs, CloudTrail records, IAM policies, source code, screenshots, or terminal output. Then compare the behavior in three cases: **normal**, **exploit**, and **post-fix**.

3) Deviation Analysis and Classification. Explain how the exploit behavior deviates from the intended rule(s). State which rule was violated, what evidence proves the violation, and why the behavior is incorrect from a security perspective. Then classify the case as one of the following:

- **Benign behavior change**
- **Accidental misconfiguration**
- **Intentional misuse / security-relevant abuse**

4) Explainable Fix and Post-Fix Validation. State clearly what assumption or logic was wrong, where the fix belongs, what exact change was made, and how your post-fix test confirms that the issue is resolved. Also confirm, where relevant, that legitimate behavior still works after the fix.

To keep submissions consistent, you must summarize this additional analysis using the **two standard tables below**. Each selected vulnerability should produce **one completed row in Table A** and **one completed row in Table B**.

Standard Summary Table Template

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
[Lesson title]	[State the intended rule(s)]	[Browser flow, code, logs, JWT, IAM policy, API response, etc.]	[Evidence showing correct behavior]	[Evidence showing vulnerable behavior]

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
[Lesson title]	[Explain why the exploit violates the intended rule(s)]	[Benign / Misconfiguration / Misuse]	[Short fix summary and location]	[Evidence that the fix worked]	[Optional timing result]

Part 10) Takeaway / Lessons Learned

Conclude with a short reflection on the lesson. You may explain:

- what security assumption caused the problem,
- why the issue was important in a serverless environment,
- and what general secure design principle would help prevent similar problems in the future.

Keep this section short and focused on the main lesson learned.

Deliverables

You are required to submit the following deliverables:

- **Written Report (PDF):** a clear, complete, and well-organized report. For **each vulnerability**, the report must include the required **10 parts** in writing, presented in a clear and consistent way so that the grader can easily follow your investigation, evidence, fix, and verification.
- **Presentation Slides (PDF or PowerPoint):** crisp and well-structured slides prepared for the project presentation. The slides should summarize your full investigation in a **short amount of time**, highlighting the most important vulnerabilities, exploitation steps, impact, fixes, and key lessons learned without unnecessary detail.
- **GitHub Repository Link:** a clean and well-organized repository containing the relevant code, configuration changes, helper scripts, screenshots, and documentation used in your project. The repository should be structured clearly enough that **a beginner can replicate your work easily** by following your instructions.
- **Demo Video Recording:** a recorded demonstration covering **all required vulnerabilities**. The demo should follow a **consistent structure** for each case, showing the vulnerability, the main reproduction steps, the observed proof, the applied fix, and the verification after remediation.

Grading Rubrics

The submission has two parts:

1. **Mandatory (10 official vulnerabilities):** you should demonstrate, explain, fix, and verify all of them.
2. **Bonus (additional vulnerabilities):** extra valid findings may increase your grade.

Important: Your report should follow the required structure for each vulnerability: **(1) Goal and Vulnerability Summary, (2) Why This Works / Root Cause, (3) Environment and Setup, (4) Reproduction Steps, (5) Evidence and Proof, (6) Fix Strategy / Probable Mitigation, (7) Code / Config Changes, (8) Verification After Fix, (9) Structured Operation and Security Analysis Tables, and (10) Takeaway / Lessons Learned.**

1) Mandatory: 10 Official Vulnerabilities (80%)

For each official vulnerability (**8% each** × **10** = **80%**), grading will follow the same structure as your report:

A. Parts 1–2: Goal, Summary, and Root Cause (1%)

- Clearly identifies the vulnerability and affected component
- Explains the security impact at a high level
- Briefly explains why the vulnerability exists

B. Parts 3–5: Environment, Reproduction, and Proof (3%)

- Includes the relevant setup details needed to understand the lesson
- Provides clear reproduction steps that can be followed by the grader
- Includes convincing proof, such as screenshots, terminal output, logs, AWS Console evidence, or API responses

C. Parts 6–7: Fix Strategy and Code / Config Changes (2%)

- Proposes an appropriate mitigation that addresses the real root cause
- Shows the actual code, policy, configuration, or logic change
- Applies the fix in the correct backend or cloud-security layer where relevant

D. Parts 8–10: Verification, Structured Operation and Security Analysis, and Takeaway (2%)

- Repeats the relevant test and shows that the vulnerable behavior no longer succeeds
- Briefly explains what changed after the fix
- Concludes with a clear security lesson or design takeaway

Scoring note:

- If you demonstrate the vulnerability but do not show a real fix, the maximum is **4/8** for that vulnerability.
- If you show a fix but do not clearly prove the original vulnerability, the maximum is **4/8**.
- If verification after fix is missing, full marks cannot be awarded for that vulnerability.
- If **Table 1** and **Table 2** from the Structured Operation and Security Analysis are missing, full marks cannot be awarded for that vulnerability.

2) Report Quality and Organization (20%)

This part evaluates the overall quality of the report across all vulnerabilities.

A. Structure and Organization (10%)

- Uses the required 1–10 part structure consistently
- Headings are clear and easy to follow
- The report is organized so that each vulnerability can be graded quickly and reliably

B. Evidence Quality and Reproducibility (5%)

- Evidence is readable, relevant, and clearly connected to the claim being made
- Commands, screenshots, logs, or AWS Console evidence are sufficient for reproduction
- Sensitive information is properly redacted where needed

C. Fix Quality and Technical Correctness (5%)

- The fix is technically sound and addresses the actual weakness
- The reported changes are specific and not vague
- Verification shows that the fix did not simply hide the symptom

3) Bonus: Additional Vulnerabilities (Up to +20%)

You can earn up to **+20%** bonus for additional vulnerabilities beyond the 10 official lessons.

Bonus scoring:

- **+2% each** for an additional *distinct* vulnerability that includes the same full reporting structure of 10 parts above:
 1. clear summary and root cause,
 2. reproduction and proof,
 3. fix strategy and actual change,
 4. verification after fix.
 5. structured operation and security analysis
- Maximum: **10** additional vulnerabilities for **+20%**

Automatic Deductions

- **-10%**: Real secrets are exposed in the report (for example valid AWS keys, valid JWTs, passwords, or session credentials).
- **-5%**: The reported fix is only cosmetic or UI-side for a backend or cloud-security issue.
- **-5%**: Evidence is too unclear to support the claimed vulnerability.
- **-5%**: The required report structure is not followed consistently.

Resources and Tools

- Official DVSA Github: <https://github.com/OWASP/DVSA>
- OWASP DVSA: <https://owasp.org/www-project-dvsa/>
- AWS Lambda: https://docs.aws.amazon.com/lambda/?icmpid=docs_homepage_featuredsvcs

Technical Support

Md. Kamrul Hossain (g202215400@kfupm.edu.sa)

You are **strongly encouraged** to attend these sessions. They are a good opportunity to discuss your progress, ask technical questions, verify whether you are on the right track, and get help with both the vulnerability analysis and the report structure. Even if you think your question is small, joining the session may save you time and help you avoid unnecessary confusion later.

The following dates are scheduled for course project related queries.

- Date: 29 March 2026, 12PM-1PM, to talk about the project.
- Date: 5 April 2026, 12PM-1PM, if you have any question about the project.

- Date: 12 April 2026, 12PM-1PM, if you have any question about the project.

Teams Meeting Link: https://teams.microsoft.com/l/meetup-join/19%3ameeting_NDJjYWRmMGYtNDIxMS00NzJjLTlkNDYtODU4NzZmZmE5ZDFj%40thread.v2/0?context=%7b%22Tid%22%3a%2229b4b088-d27d-4129-b9f9-8637b59ea4b3%22%2c%220id%22%3a%2229a717896-fa32-44e3-8239-3cb3d2a86d2d%22%7d

Team Structure

This project may be completed either **individually** or in a **team of up to 4 students**. Students are free to choose the team size that best matches their working style; however, the expected **depth, breadth, and polish** of the submission should scale with team size. In general, larger teams are expected to produce a **richer and more technically mature investigation**, including stronger analysis, clearer evidence, better fixes, more thorough verification, and, where possible, additional well-justified findings.

Team expectations.

- A team of **1** student must still complete **all required project components**. Working individually does **not** reduce the scope of the assignment.
- A team of **2–4** students is expected to demonstrate correspondingly greater technical depth, clearer organization, and stronger overall completeness.
- Work may be divided among team members, but **every member must contribute meaningfully** and must understand the full project.
- Each team member should be able to explain any part of the submission, including the vulnerability logic, exploit steps, evidence, remediation, verification, and major design or implementation decisions.

Use of AI tools.

- You may use AI tools to support learning, improve clarity, help organize ideas, assist with debugging, and strengthen your presentation of the work.
- However, you remain fully responsible for the **correctness, originality, and understanding** of everything you submit.
- You must be able to explain and justify all technical content in your report, slides, repository, and demo.

Collaboration boundaries and academic integrity.

- High-level discussion of concepts is acceptable, but it is **strictly prohibited** to copy code, fixes, write-up text, screenshots, tables, analysis, or ideas from classmates or other teams.
- Your submission must reflect **your own work or your own team's work**.
- Any copied or closely imitated material may be treated as an academic integrity violation.

Recommendation to start early. This project involves deployment, experimentation, debugging, documentation, fixing, and verification. You are therefore **strongly encouraged to start early**. Early progress will give you more time to understand the system, avoid unnecessary pressure, improve the quality of your analysis, and produce a stronger final submission.

Why this project matters. A serious effort on this project will help you develop practical skills in serverless security, cloud architecture, vulnerability analysis, secure coding, systematic reporting, and technical communication. These are valuable skills for future work in cybersecurity, cloud engineering, secure software development, and graduate-level study or research.

Final Note

This project is designed to push you beyond passive learning and into real technical investigation. You will deploy, test, break, analyze, fix, verify, and explain security issues in a realistic cloud environment. That is not always easy, but it is exactly the kind of experience that helps you grow.

If you start early, stay organized, and work carefully, you will learn much more than how to reproduce a few vulnerabilities. You will practice thinking systematically, communicating clearly, and approaching security problems with both technical depth and professional discipline.

Good luck — and remember: in this course, the goal is not only to hack the system, but to understand it deeply enough to secure it.